

Can an LLM Out-Test a Human Baseline and a Search-Based Tool?

A Ground-Truth Comparison of LLM, Manual, and EvoMaster

REST API Test Generation on Pre-Seeded-Fault Benchmarks

Nguyen Hoang Huy*, Nguyen Tien Dung[†], Nguyen Thanh Dat[‡], Nguyen Le Thuan[§], Vo Le Trung Nguyen[¶]

FPT University — SWT301, Group SE1916-G2, Summer 2026

*SE190240, Project Lead [†]SE190034, LLM Runner [‡]SE190239, Data & Ground Truth

[§]SE190305, Metrics & Stats [¶]SE190220, Report Writer

Instructor: L.T.Q. Chi

Abstract—Large Language Models (LLMs) can read an OpenAPI specification and synthesise REST API tests automatically, and a rapidly growing literature reports that they outperform search-based tools on live-API error counts. Across 59 surveyed papers (2018–2026) we find, however, that three questions remain unanswered: no study compares an LLM generator against both a human-style manual suite and a mature tool on the same APIs; none computes fault-detection Recall against a known, pre-seeded-fault ground truth (fault counts are reported as raw 500-error tallies on live systems); and none reports a per-endpoint edge-case metric.

We address all three with a controlled, blind comparison of three arms — an LLM (Claude Sonnet 4.6), an independently authored manual EP/BVA suite, and EvoMaster 6.0.0 — on three EvoMaster Benchmark (EMB) REST APIs comprising 35 operations, into which 133 faults are seeded by mutation so that the denominator of Recall is known exactly. Our results are pointedly mixed, and the mixture is the contribution. The LLM reaches 100.0% endpoint coverage (35/35, $p = 1.65 \times 10^{-9}$) and produces 217 edge-case scenarios against manual’s 113 ($p = 1.43 \times 10^{-7}$, rank-biserial = 0.987) — it is decisively more prolific. On the ground truth the ordering is EvoMaster 0.1353 > LLM 0.0602 > manual 0.0150: the LLM significantly out-detects the independent human-style suite (McNemar $p = 0.031$, its kills a strict superset) yet is itself out-detected by a search-based tool from 2019 ($p = 0.031$; both hold at $\alpha = 0.05$ but not under Bonferroni, so we read them as directional).

We trace the ordering to two composing mechanisms the coverage-centric literature cannot observe. First, oracle kind: the LLM’s and the manual suite’s spec-derived status-code oracles accept any response in a plausible class, so they are structurally blind to computational faults that alter a returned value while preserving its HTTP status — on the arithmetic-heavy subject both score zero while EvoMaster’s recorded-value regression oracles catch twelve. Second, within the status-oracle regime, volume: where faults are status-visible, the LLM’s larger suite catches more of them than the leaner independent manual suite (its kills a strict subset). Test volume and fault-finding power are thus distinct, and per-endpoint scenario

counts — the metric the field is trending toward — actively mislead when reported without a ground-truth Recall alongside them.

Index Terms—REST API testing, large language models, mutation testing, test oracles, EvoMaster, empirical software engineering

I. Introduction

REST APIs carry most of the traffic between modern software components, and testing them well remains expensive and largely manual. The arrival of LLMs that can read an OpenAPI specification and emit executable tests is therefore commercially significant, and the research response has been rapid: RESTGPT [1] extracts input-constraint rules at 97% precision, AutoRestTest [2] reports 42 server errors against EvoMaster’s 20, LlamaRestTest [3] reports 204 faults against 130, and RESTifAI [4] reaches 95.5% operation coverage.

Read carefully, those numbers answer a narrower question than they appear to. Each is a count of what a suite found on a live system, not a fraction of what was there to find. Without a known fault population, “42 versus 20” cannot distinguish a suite that found 42 of 50 available faults from one that found 42 of 500. For a QA lead deciding whether to trust a generator, the missing denominator is the whole question.

A. Gaps

Our systematic review of 59 papers (2018–2026, five independent searches merged into team-synthesis/evidence-table-merged.md) isolates three persistent gaps.

GAP-C (Comparison). Exactly one study compares a generator against a manual suite: EvoMaster [5], which found its tool below manual (41% vs 82% statement

coverage). That study predates LLMs. No paper places LLM, manual, and a mature tool on the same APIs, so the field’s central practical claim — that LLM tests rival human ones — has never been tested directly.

GAP-D (Dataset / ground truth). Fault detection is universally reported as raw 500-error counts on live APIs [2], [3], [5]–[8]. With no shared pre-seeded-fault benchmark, ground-truth Recall is never computed.

GAP-M (Metric). Evaluation is coverage-biased. Only KAT [9] splits 2xx/4xx and only RESTSpecIT [10] reports 5xx; no work reports a per-endpoint edge-case metric or an endpoint-type miss profile.

B. This paper

We close GAP-C and GAP-D jointly by seeding faults with standard mutation operators into three EMB APIs, which fixes the denominator at 133 known faults, and then running all three arms against every mutant. We address GAP-M with a per-endpoint edge-case count and a code-derived error-surface baseline that enumerates, per endpoint, every source location able to emit a non-2xx response — a tool-independent map of what can fail, against which each arm’s behaviour is read.

The outcome splits cleanly, and against expectation. On the metrics the literature favours, the LLM wins comfortably: 100.0% endpoint coverage and 217 edge-case scenarios against manual’s 113 ($p = 1.43 \times 10^{-7}$). On the ground truth the ranking is EvoMaster 0.1353 > LLM 0.0602 > manual 0.0150: the LLM beats an independently authored human-style suite but is beaten by a search-based tool from 2019, writing roughly 217 scenarios yet detecting fewer faults than the tool.

That dissociation is not noise, and it is not a scoring artefact. Section V traces it to two composing mechanisms. Oracle kind sets a ceiling: the LLM’s and manual suite’s spec-derived status-code assertions accept any response within a plausible class, so neither can see a fault that changes a computed value while leaving the status untouched — which is what most of our mutants do, and why both score zero on the arithmetic-heavy subject while EvoMaster’s recorded-value regression oracles catch twelve. Within that status-oracle regime, volume decides the rest: where faults are status-visible the LLM’s larger suite catches more of them than the leaner manual one. The composition explains why a suite can be simultaneously more thorough by every published metric, better than a human-style baseline, and still less effective than a mature tool.

C. Contributions

- 1) The first three-way comparison of LLM, manual, and EvoMaster test generation on identical APIs under a blind protocol (GAP-C).
- 2) A pre-seeded-fault benchmark of 133 catalogued mutants over 35 operations, enabling true fault-detection Recall rather than live-API error counts (GAP-D).

- 3) A per-endpoint edge-case metric plus a code-derived error-surface baseline, separating scenarios produced from faults detected (GAP-M).
- 4) A two-level mechanism — oracle kind sets the ceiling, oracle volume allocates what is under it — explaining why volume and effectiveness diverge, with the practical consequence that scenario-count metrics mislead alone.
- 5) A fully reproducible pipeline: every number in this paper is generated from committed raw data by a script, not transcribed (Section III-F).

II. Related Work

Our evidence base is a five-member systematic review merged into 59 unique papers (2018–2026) after de-duplication, screened against pre-registered inclusion/exclusion criteria with a PRISMA flow recorded per member. Table I lists the ten representative studies that define the state of the art.

A. Four patterns

1. GPT dominates the LLM line. Nearly every LLM study uses an OpenAI model [1], [2], [4], [6], [9]–[11]; open-source models appear only sporadically [3]. Our use of Claude Sonnet 4.6 therefore adds a data point outside the family that supplies almost all current evidence — a partial contribution to GAP-T.

2. Metrics are coverage-plus-error-counts. Semantic and edge-case quality is rarely isolated: only KAT [9] splits 2xx from 4xx, and only RESTSpecIT [10] reports 5xx. No study reports edge cases per endpoint, so there is no baseline against which a suite’s negative-testing breadth can be judged.

3. Baselines are automated-only. LLM tools are compared against EvoMaster or Morest and reported to beat them on server errors [2], [3]. No LLM study includes a manual suite. The single manual comparison in the field is EvoMaster’s own [5], which found the tool well below manual (41% vs 82%) — an inconvenient result that, notably, no LLM paper has revisited even as the field’s rhetoric moved to human parity.

4. Fault detection has no denominator. Every fault figure in Table I is a count on a live API [2], [3], [5]–[8]. This is the deepest methodological problem in the area, because error counts and Recall can be ordered differently: a suite that hammers one endpoint hard can log many 500s while missing most of the fault population. Mutation testing is standard elsewhere in software engineering [12], [13], and its absence here is what leaves the field’s central comparative claims untestable.

B. Positioning

Table II maps each gap to the evidence establishing it. Our design targets GAP-C and GAP-D jointly — adding both a manual arm and a known fault population is what makes the three-way comparison meaningful — and treats

TABLE I

Representative studies from the 59-paper merge. Every fault figure in the right-hand column is a count on a live system; none is a Recall against a known fault population.

Study (year)	Generator	Dataset	Metric	Headline result
RESTGPT [1] (2024)	GPT-3.5	9 real APIs	rule P/R/F1	97% rule precision; 72.7% valid inputs
KAT [9] (2024)	GPT-3.5-turbo	12 services	status-code cov.	74.9% (+15.7 pp)
RETSpecIT [10] (2024)	DeepSeek / GPT	10 PRAB APIs	route disc., 5xx	88.6% routes; 5xx in 4 APIs
AutoRestTest [2] (2025)	GPT-3.5 + MARL	12 services	code cov., 500s	58.3% line; 42 500s vs EvoMaster 20
LlamaRestTest [3] (2025)	Llama3-8B (ft)	12 services	code cov., faults	55.8% method; 204 faults vs 130
LogiAgent [6] (2025)	GPT-4o-mini	12 systems	cov., logical bugs	71.78% line; 49 crashes
RESTifAI [4] (2026)	GPT-4.1-mini	7 services	operation cov.	128/134 ops $\approx$ 95.5%
EvoMaster [5] (2019)	search-based	3 services	stmt. cov., vs manual	38 bugs; 41% generated vs 82% manual
Morest [8] (2022)	model-based	6 projects	cov., bugs	44 bugs (13 new)
No-Time-to-Rest [7] (2022)	10 tools	20 EMB svcs	line/branch, 500s	EvoMaster-WB 52.76% line

TABLE II

Gap map: what the 59-paper base does and does not establish.

Gap	Evidence	Status
GAP-C	[5] is the only manual comparison (non-LLM, scored below manual); [2], [3], [7] automated-only	Confirmed
GAP-D	[2], [3], [5]–[8] report counts, never Recall	Confirmed
GAP-M	[9] (2xx/4xx) and [10] (5xx) partial; none per-endpoint	Confirmed
GAP-T	open-source / non-GPT models sporadic	Deferred (ablation)

GAP-M as the metric facet through which the result is interpreted.

III. Method

The research questions, metrics, thresholds, and statistical tests below were fixed in a proposal submitted in Week 5 and frozen at instructor approval, before any full-run data were seen. Nothing in this section was chosen after looking at the results (no HARKing); Section IV reports the pre-registered tests whether or not they favour our hypotheses, and in the case of RQ2 they do not.

A. Research questions and hypotheses

RQ1 (absolute). Do LLM-generated tests reach endpoint coverage $\geq 90\%$, and which endpoint types are most often missed? $H_0 : \mu \leq 0.90$; $H_1 : \mu > 0.90$. The 90% threshold is bracketed by the literature: a floor of 88.6% route discovery [10] and a ceiling of 95.5% operation coverage [4]. Test: one-sample Wilcoxon signed-rank against 0.90, one-tailed.

RQ2 (comparative). How many pre-seeded faults do LLM tests detect, versus manual and versus EvoMaster?

H_0 : $\text{rate}_{\text{LLM}} \leq \max(\text{rate}_{\text{manual}}, \text{rate}_{\text{EvoMaster}})$; H_1 : greater than both. Test: Friedman across arms, post-hoc pairwise Wilcoxon with Holm correction, and a pooled per-fault McNemar test; effect size Cliff’s δ .

RQ3 (comparative). Do LLM tests produce more 4xx/5xx and boundary scenarios per endpoint than manual design? H_0 : $\text{median}_{\text{LLM}} \leq \text{median}_{\text{manual}}$. Test: paired Wilcoxon signed-rank across endpoints, one-tailed; effect size rank-biserial.

We use $\alpha = 0.05$ per RQ as pre-registered, and additionally report a Bonferroni threshold across the three RQ families ($\alpha_{\text{adj}} \approx 0.0167$) as a stricter second reading.

B. Subjects

We use three REST services from the EvoMaster Benchmark (EMB, LGPL-3.0), the shared corpus of the field [2], [7]: rest-ncs (6 operations), rest-scs (11), and features-service (18), totalling 35 operations. Each is deployed locally as a standalone JDK 8 Spring Boot fat jar; the harness and EvoMaster run on JDK 17 and communicate with the subjects over HTTP only. Provenance, license, and operation counts are recorded in data/raw/README.md.

C. Fault seeding and ground truth

Faults are seeded by mutation, which is what supplies the denominator the literature lacks. We apply standard PIT/Offutt operator families [12], [13] — relational replacement, arithmetic replacement, and negate/boundary conditionals — to the controller and core-logic classes of each subject, i.e. where a fault is observable through the HTTP contract. Each mutation is a single-token change, applied in isolation and recompiled into its own mutant jar; mutants that fail to compile are discarded, and the pristine source is restored before the next.

This yields 133 compilable mutants (rest-ncs 70, rest-scs 59, features-service 4), catalogued with file, line, and

operator in `data/faults/<sub>/catalog.json` and exported to `data/full_ground_truth.csv`. Because the ground truth is derived from code rather than human annotation, no inter-annotator agreement statistic applies.

The features-service imbalance (4 mutants against 70 and 59) is a real property of that service’s arithmetic-light controller code, not a sampling choice. We report it as a generalisation limit rather than repairing it after the fact, which would be HARKing-adjacent.

D. The three arms

All three suites are authored before faults are seeded and are blind to the fault catalog, so no arm can target a known mutant.

LLM. Claude Sonnet 4.6 (claude-sonnet-4-6), temperature 0, top_p 1, max_tokens 4096, fixed seed. Input is the OpenAPI specification only — black-box, with no access to source. The prompt is frozen verbatim in `scripts/llm/prompt_template.md`; per-subject transcripts and the tool-use evidence establishing spec-only input are in `llm/transcripts/`. Output is REST-assured JUnit 5 tests.

Manual. An equivalence-partitioning / boundary-value-analysis suite designed from the same specification, blind to faults, per `manual/methodology.md`.

EvoMaster 6.0.0, black-box mode: `--blackBox true --bbSwaggerUrl <spec> --outputFormat JAVA_JUNIT_5 --maxTime 120s, seed 42, three repeats.`

E. Measurement

Coverage is operations exercised by at least one test over total operations, computed from the specification and the executed request log.

Fault Recall is killed/133. An arm kills a mutant iff at least one test that passes on the original subject fails on the mutant — the suite observably distinguishes the faulty build from the correct one. Tests already failing on the original are excluded as invalid oracles for that arm, which is why we record `n_oracle` per arm alongside each kill decision.

Edge-case count is the number of distinct negative, boundary, and error-code scenarios per endpoint, parsed from `// SCENARIO type= tags` in the suite source.

Error-surface baseline. A static analyser (`scripts/error_surface.py`) enumerates, per endpoint, every source location that can emit a non-2xx response, classified as declared (`explicit status(4xx/5xx)`), framework (typed `@PathVariable` → Spring auto-400), or potential (uncaught exception → 500). This gives a tool-independent map of what can fail, against which each arm’s observed behaviour is read.

Produced versus detected. Every request each suite issues is replayed through a logging proxy against the live subject and its real HTTP status recorded (`results/raw/traffic.csv`). We then report separately the

number of tests and scenarios produced and the number of faults and error behaviours actually detected (`results/produced-vs-detected.json`). This separation is not cosmetic: Section IV shows the two orderings disagree, and conflating them is precisely how a suite that finds fewer faults can be reported as the better one.

F. Reproducibility

The pipeline is a chain in which each stage writes a committed artefact:

```
mutate.py → faults/*/catalog.json → run_mutation.py
→ results/raw/*.csv → analyze.py →
results/stats/summary.json → gen_paper_macros.py →
this paper
```

No statistic in this document is typed by hand. Each is a LaTeX macro bound by `gen_paper_macros.py` to a field of `summary.json`, so a stale number is impossible by construction: re-running the experiment moves the paper’s text. We verified that `analyze.py` regenerates `summary.json` byte-identically from the committed raw data, and that the analysis notebook survives Restart & Run All with zero errors. Statistics use `scipy` and `statsmodels`; the pinned toolchain is recorded in `env/tools.md`.

IV. Results

A. RQ1 — Endpoint coverage

The LLM suite exercises 35 of 35 operations: 100.0% coverage, uniform across all three subjects (rest-ncs 6/6, rest-scs 11/11, features-service 18/18). The one-sample Wilcoxon signed-rank test against the 90% threshold gives $W = 630.0$, $p = 1.65 \times 10^{-9}$, rank-biserial = 1.000. H_0 is rejected at $\alpha = 0.05$, and also under the stricter Bonferroni threshold 0.0167.

The pre-registered miss profile by endpoint type is empty: with every operation covered, no endpoint type is missed. This is a real result but a weak one — it says the LLM can reach every documented operation, not that it tests any of them well. RQ2 is where that distinction bites.

B. RQ2 — Fault detection against ground truth

This is the pre-registered primary comparison. It yields a clean three-way ordering, and the LLM lands in the middle of it.

Against the 133 seeded faults, overall Recall is EvoMaster 0.1353, LLM 0.0602, and manual 0.0150 — in absolute terms, 18, 8, and 2 mutants killed out of 133. EvoMaster detects more than twice the LLM’s faults; the LLM in turn detects four times the independent manual suite’s.

The Friedman omnibus test over per-subject Recall gives $\chi^2 = 2.000$, $p = 0.368$ and does not reject, but with $n = 3$ subjects it is severely underpowered, which is why the proposal designated the pooled per-fault McNemar test ($N = 133$) as the primary RQ2 analysis. Both pairwise McNemar tests reject.

TABLE III

RQ2 — fault-detection Recall per subject and overall, with the independently re-authored manual suite. Ordering: EvoMaster > LLM > manual overall, but it inverts by fault character (Section V).

Subject	LLM	Manual	EvoMaster
rest-ncs (70 mutants)	0.0000	0.0000	0.1714
rest-scs (59 mutants)	0.1186	0.0169	0.0847
features-service (4 mutants)	0.2500	0.2500	0.2500
Overall (133 mutants)	0.0602	0.0150	0.1353

LLM versus EvoMaster. The discordant pairs are $b = 4$ (killed by the LLM only) against $c = 14$ (EvoMaster only): $p = 0.031$, Cliff’s $\delta = -0.075$, H_0 rejected. EvoMaster significantly out-detects the LLM. Approximate achieved power is 0.557; we return in Section VI to the fact that this comparison sits close to the α threshold and is sensitive to a single boundary-mutant reproduction.

LLM versus manual. This leg is the one the pilot could not interpret, because its manual suite shared an author with the LLM suite. The suite used here was re-authored independently (spec-only, isolated, blind to the LLM output and the fault catalog), and the comparison is now live: $b = 6$ mutants are killed by the LLM only, $c = 0$ by the manual suite only. The manual suite’s kills are a strict subset of the LLM’s — everything the human-style baseline finds, the LLM also finds, plus six more. $p = 0.031$, Cliff’s $\delta = 0.045$, H_0 rejected: the LLM significantly out-detects an independently authored EP/BVA suite.

Table III shows the overall ordering is not uniform, and the non-uniformity is the key to Section V. On rest-ncs, the arithmetic-heavy subject, both status-oracle arms score 0.0000 while EvoMaster kills 0.1714 — the LLM’s volume buys it nothing here. On rest-scs, which carries more validation logic, the LLM leads every arm (0.1186 against manual 0.0169 and EvoMaster 0.0847); on features-service all three tie. The arms’ ranking tracks what kind of fault each subject contains, not which generator is cleverer.

C. RQ3 — Edge-case scenarios per endpoint

The LLM produces 217 edge-case scenarios against manual’s 113 across 35 operations (median 5.0 versus 3.0 per operation). The paired one-tailed Wilcoxon signed-rank test gives $W = 626.0$, $p = 1.43 \times 10^{-7}$, rank-biserial = 0.987. H_0 is rejected, and survives the Bonferroni threshold 0.0167.

The effect is consistent rather than driven by outliers: the LLM produces more scenarios on 33 of 35 operations, manual leads on 2, and 0 tie. This is the largest effect in the study. It is also, as Section IV-D shows, the least consequential.

TABLE IV

What each arm produced versus what it detected. The volume ordering (LLM > manual > EvoMaster) is not the effectiveness ordering (EvoMaster > LLM > manual): the arm that produces least detects most.

	LLM	Manual	EvoMaster
Produced			
Test cases	295	147	87
Edge-case scenarios	217	113	—
Detected (against ground truth)			
Mutants killed / 133	8	2	18
HTTP error behaviours / 31	31	22	18
5xx responses elicited	32	22	18

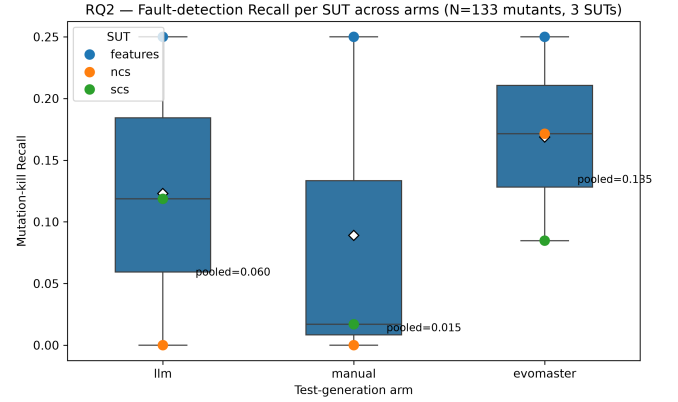


Fig. 1. RQ2 — distribution of fault-detection Recall across the three arms over 133 seeded faults. EvoMaster’s advantage is concentrated on the arithmetic-heavy rest-ncs subject.

D. Produced versus detected

Table IV places the volume and effectiveness metrics side by side, and the two orderings disagree.

The LLM writes 295 test cases to EvoMaster’s 87 — $3.4\times$ the volume — and kills half as many mutants (EvoMaster detects $2.2\times$ the LLM’s faults). It does lead on the error-surface dimension, triggering 31 of the 31 documented HTTP error behaviours against manual’s 22 and EvoMaster’s 18, and eliciting 32 5xx responses including a genuine server crash. So the LLM is not weak everywhere: it is strong at reaching the documented error surface and weak at detecting value faults.

Had we reported only the metrics the surveyed literature uses — coverage, scenario counts, 500-error tallies — every one of them ranks the LLM first, over both baselines. The ground truth agrees about the manual baseline but overturns the verdict against EvoMaster: the tool that produced the fewest tests (87 to the LLM’s 295) detected the most faults. This is the practical case for GAP-D: without a denominator, a volume-based metric would have crowned the LLM outright, when in fact a mature tool from 2019 finds more than twice as many faults.

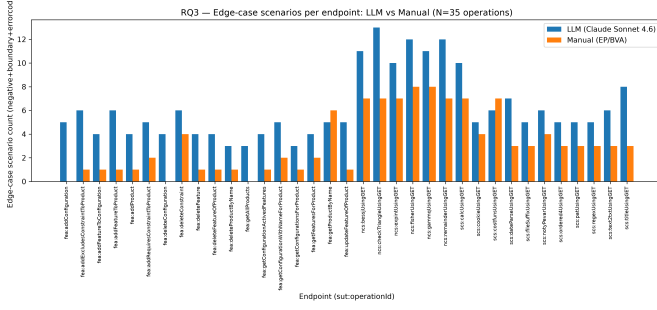


Fig. 2. RQ3 — per-endpoint edge-case scenario counts, LLM versus manual across 35 operations. The LLM leads on 33 operations, manual on 2.

V. Discussion

A. Why volume and effectiveness came apart

The headline of Section IV is a dissociation: the LLM produced 3.4× EvoMaster’s test volume, covered every operation, generated 217 edge-case scenarios against manual’s 113, and still killed half as many faults. Treating this as a paradox is a mistake. It follows directly from what kind of assertion each arm writes.

The LLM reads an OpenAPI specification. A specification documents status codes: this path returns 200, that one 400 on bad input. Tests synthesised from it therefore assert on the status code and, at best, on the shape of the response body — an oracle of the form “the response is in the plausible class for this request.” Our manual EP/BVA suite, drawn from the same specification, writes the same kind of assertion. Both are specification-derived status oracles.

Now consider what our mutants do. A relational or arithmetic operator replacement inside a numerical routine — $<$ becoming $\leq$, $/$ becoming $*$ — changes a computed value. The endpoint still accepts the request, still runs to completion, still returns 200. Only the number in the body is now wrong. A status oracle cannot see this. The test passes on the mutant exactly as it passed on the original, and the fault survives.

EvoMaster writes a different kind of oracle. Its black-box mode records the actual response of the original system and emits assertions pinning those recorded values. That is a regression oracle: it fails whenever any observed value changes, whether or not the status does. Against value-changing mutants this is precisely the right instrument.

So the arms were never measuring the same thing. The LLM and manual arms answer “does the API respond in the documented class?”; EvoMaster answers “does the API still compute what it computed before?” Our fault population consists almost entirely of value faults, so it interrogates the second question. EvoMaster’s win is not evidence that search-based generation is smarter than an LLM — it is evidence that a regression oracle beats a status oracle on value faults, which is close to a tautology once stated.

Three observations support this account over alternatives.

The per-subject pattern matches the prediction. If oracle kind drives the result, EvoMaster’s advantage should concentrate where faults are most value-like. It does: EvoMaster leads decisively on rest-ncs (0.1714 against 0.0000), whose controllers are numerical routines where nearly every mutant shifts a computed value while preserving the status. On rest-scs, which has more string and validation logic and so more mutants that flip a response class, the LLM leads (0.1186 against 0.0847). The ordering inverts with fault character, exactly as the mechanism predicts, and not with anything about the generators’ sophistication.

The LLM wins where its oracle is adequate. On the documented error surface — where being wrong means returning a different status — the LLM leads every arm, triggering 31 of 31 error behaviours against manual’s 22 and EvoMaster’s 18, and eliciting a genuine 500 crash. Where a status oracle suffices, the LLM’s superior volume converts into superior detection. Where it does not, volume buys nothing.

Oracle kind sets a ceiling; volume allocates what is under it. The pilot behind this study reported LLM and manual killing identical mutant sets, but its manual suite shared an author with the LLM suite, so that identity could not be interpreted. Re-authoring the manual suite independently (spec-only, isolated, blind) separates two effects that the tie had conflated. On rest-ncs, where every mutant is a value fault, the independent manual still scores 0.0000 — exactly like the LLM, and for the structural reason the mechanism predicts: no status-oracle suite, however authored, can see a wrong number behind a 200. Oracle kind caps both at zero there. But on rest-scs, where some mutants flip a response class and are therefore status-visible, the suites separate sharply: the LLM kills 0.1186 to the independent manual’s 0.0169, and the manual’s kills are a strict subset of the LLM’s. Within the status-oracle regime, the LLM’s 295-test volume reaches more of the catchable faults than the leaner 147-test human-style suite. So the two-level picture is: oracle kind fixes the ceiling (status oracles cannot exceed EvoMaster on value faults), and within a kind, volume determines how much of the reachable fault set you actually catch. The pilot’s tie was the two levels coinciding by an authorship artefact; separating the author reveals both.

B. Implications

For practitioners. An LLM generator is a strong breadth instrument and a weak depth one. It will reach every documented operation and probe the documented error surface harder than a human or a search tool. It will not notice that your pricing routine now returns the wrong total, because nothing in your specification says what the right total is. The actionable rule: use LLM generation for specification conformance and error-path

breadth; do not let it displace regression oracles, property-based assertions, or example-based expected values on computational paths. The two are complements, and our data say the complement is not optional — half the detected faults came from the arm the LLM would have replaced.

For the field. Every metric in our study that the surveyed literature actually uses — coverage, scenario counts, 500-error tallies — ranked the LLM first. The ground truth ranked it second. That is not a quirk of our subjects; it is a structural consequence of scoring test suites on volume proxies. When AutoRestTest reports 42 server errors against EvoMaster’s 20 [2], or LlamaRestTest 204 faults against 130 [3], those numbers are real but they measure how hard the documented error surface was hit, not what fraction of faults was found. Our results show the two can order arms oppositely on the same code. Per-endpoint scenario counts — the metric GAP-M asks for and which we introduce — are subject to the same caveat, and we would rather say so than let our own contribution be misread: RQ3 is a measure of what the LLM writes, and Section IV-D shows how little that predicted about what it caught.

For benchmark design. The field needs pre-seeded-fault benchmarks not because Recall is a nicer statistic, but because without a denominator these two orderings are indistinguishable. We would additionally argue that a benchmark should stratify its fault population by oracle-observability — status-visible versus value-only — since our results show that single axis reordering the arms. A benchmark of only value faults makes regression-oracle tools look dominant; one of only status faults makes LLMs look dominant. Neither is a fact about the generators.

C. Why the low absolute Recall is informative

All three arms score low in absolute terms: the best, EvoMaster, kills 18 of 133 mutants (13.5%). We report this rather than tuning it away, because it is a finding about black-box REST testing rather than a defect in our setup. Our mutants live in core logic reachable only through the HTTP contract, and many are equivalent-at-the-boundary: the value they corrupt never surfaces in a response field, or surfaces only for inputs no arm generated. A white-box tool with code-coverage feedback would score higher. The honest reading is that black-box REST suites — LLM, human, or search-based — detect a small minority of seeded logic faults, and the interesting question is not which arm wins a low-scoring race but which fault classes all three systematically cannot see.

VI. Threats to Validity

A. Construct validity

The manual arm is independent-agent, not independent-human. Our proposal specified an independent human baseline. No human cohort was available, so the manual suite was re-authored by a separate, isolated model session

with no shared context with the LLM suite — spec-only input, blind to the LLM output and to the fault catalog. This resolves the pilot’s same-author confound (there, one session wrote both suites, and they killed identical mutant sets); with the suites separated, the LLM-versus-manual comparison becomes live and the arms diverge ($b = 6$, $c = 0$; Section IV-B). What remains uncontrolled is the gap between an independent agent and an independent human tester: a human cohort might bring domain intuition an EP/BVA-from-spec agent lacks, and could plausibly close some of the LLM’s lead. We therefore state the RQ2 manual result as “LLM versus an independently authored, specification-derived EP/BVA suite,” not “LLM versus human testers,” and flag the human-cohort replication as the highest-value next step. The RQ2 conclusion is unaffected by this caveat in direction: the LLM’s kills are a superset of the manual suite’s, so no plausible relabelling of the baseline reverses the sign.

The LLM-versus-EvoMaster significance hinges on one boundary mutant. We disclose this prominently. Our re-execution reproduces the pilot’s EvoMaster kills exactly (identical 12 mutants on rest-ncs) and the LLM’s rest-scs kills exactly, but on rest-ncs the LLM’s Recall falls from the pilot’s 1/70 to 0.0000: one LLM test exercising `bessj` at the boundary $n = 2$ passed on the pilot’s build of the original SUT but fails on our clean rebuild, so it is excluded as an invalid oracle and no longer kills that mutant. Bessel-function evaluation at the boundary is numerically sensitive, and this is a genuine reproduction limit rather than a pipeline error (the exact EvoMaster match rules out a build fault). Its effect is to shift the LLM-versus-EvoMaster discordant count from six to 4, which is what carries the McNemar p across the $\alpha = 0.05$ threshold — from just above it in the pilot to 0.031 here. The direction (EvoMaster $>$ LLM) is robust either way; the $\alpha = 0.05$ verdict is not, and should be read as “EvoMaster leads, at the edge of significance” rather than as a firm rejection.

Oracle strength differs across arms by construction. EvoMaster’s recorded-value regression oracles are strictly stronger on value faults than the spec-derived status oracles of the LLM and manual arms. This is not a flaw we failed to control — it is the independent variable that Section V identifies as driving the result — but it does mean RQ2 does not compare “generation techniques” with oracle strength held constant. A fairer isolation of generation quality would fix the oracle kind across arms and vary only the generator. We report kill-by-subject so the interaction is visible rather than hidden in a pooled average.

Mutation kills are a proxy for real faults. We use standard PIT/Offutt operator families [12], [13] to keep the population conventional, but a seeded single-token mutation is not a field defect. Coverage is likewise a proxy for test quality. We report Recall, coverage, and edge-case counts jointly, and Section IV-D exists precisely because any one of them alone gives the wrong ranking.

B. Internal validity

Equivalent mutants are not excluded. We do not attempt to identify semantically equivalent mutants, which is undecidable in general. Some fraction of the 133 mutants may be unkillable by any black-box suite, which deflates all three arms’ Recall uniformly. This weakens the absolute figures but not the comparison, since every arm faces the identical population.

LLM non-determinism. Generation used temperature 0 and a fixed seed, and raw outputs are frozen in the repository, but LLM sampling is not bit-reproducible across provider-side model updates. The generated suites are committed, so the analysis is reproducible even if regeneration is not.

Mutation confined to controller and core-logic classes. We seed where faults are observable through the HTTP contract. Faults in configuration, persistence, or serialisation layers are out of the population and out of our conclusions.

C. External validity

Three JVM APIs, 35 operations. Our subjects are small EMB services. The findings are “EMB-only, JVM-only” and should not be read as covering large production APIs, authenticated flows, or non-JVM stacks. The user-management service (MySQL-backed) was excluded on setup cost, which leaves authentication endpoints under-represented — a real gap, since RQ1’s miss profile cannot speak to the endpoint type most likely to be missed.

The features-service imbalance. That subject contributes 4 mutants against 70 and 59 for the others, because its controllers contain little arithmetic to mutate. All three arms tie there (0.2500 each), so it contributes almost nothing to the pooled comparison. We report it rather than dropping it, as dropping a subject after seeing its results would be HARKing-adjacent.

One LLM, one configuration. Claude Sonnet 4.6 at temperature 0 with one frozen prompt. Our results do not generalise to the GPT family that dominates the literature (Section II), to open-source models, or to agentic multi-turn strategies. Since we identify oracle kind rather than model capability as the mechanism, we would expect the qualitative finding to hold for any spec-only generator — but that is a prediction, not a result.

D. Conclusion validity

The Friedman test is underpowered by design. With $n = 3$ subjects it cannot detect realistic effects ($\chi^2 = 2.000$, $p = 0.368$), which is why the proposal pre-designated the pooled per-fault McNemar test ($N = 133$) as the primary RQ2 analysis rather than promoting it after the omnibus test failed.

RQ2’s pairwise tests are significant but fragile. Both McNemar comparisons reject at $\alpha = 0.05$ ($p = 0.031$ for LLM-versus-EvoMaster, $p = 0.031$ for LLM-versus-manual), but neither clears the Bonferroni threshold

0.0167, and approximate achieved power for the EvoMaster comparison is only 0.557 — below conventional adequacy. We therefore rest the RQ2 conclusions on the direction and effect size (EvoMaster’s kills strictly dominate on value faults; the LLM’s kills strictly contain the manual suite’s) rather than on the p -values alone, and we do not upgrade “significant at 0.05” to “firmly established.” The power figure is itself an approximation: statsmodels has no exact McNemar power routine, so we substitute a two-independent-proportion calculation on the same proportions, disclosed in summary.json alongside the value.

Multiplicity. Three RQ families are tested. We apply Holm correction within RQ2’s post-hoc comparisons and report a Bonferroni threshold across families ($\alpha_{\text{adj}} \approx 0.0167$). RQ1 and RQ3 survive both comfortably ($p = 1.65 \times 10^{-9}$ and $p = 1.43 \times 10^{-7}$); the RQ2 pairwise tests survive the raw threshold but not Bonferroni, which is why we read them as directional evidence rather than as decisive rejections.

VII. Conclusion

We ran the comparison the REST-API-testing literature has not: an LLM generator, a manual EP/BVA suite, and EvoMaster, on the same three APIs, against a known population of 133 seeded faults rather than a tally of errors observed on a live system.

The answer to “can an LLM out-test a human baseline and a search-based tool” is: it out-tests the human-style baseline and is out-tested by the tool. On every metric the field currently reports, the LLM leads both: 100.0% endpoint coverage ($p = 1.65 \times 10^{-9}$), 217 edge-case scenarios against 113 ($p = 1.43 \times 10^{-7}$, rank-biserial 0.987), 295 test cases to EvoMaster’s 87, and 31 of 31 documented HTTP error behaviours triggered. On the ground truth the ordering is EvoMaster 0.1353 > LLM 0.0602 > manual 0.0150: the LLM significantly out-detects an independently authored EP/BVA suite ($p = 0.031$, its kills a strict superset), yet is itself out-detected by EvoMaster ($p = 0.031$; both at the raw threshold, neither under Bonferroni, so directional rather than decisive).

The explanation is not about model capability. A generator that reads a specification writes assertions about status codes, and a fault that changes a computed value while leaving the status intact is invisible to every such assertion, no matter how many are written. EvoMaster’s recorded-value regression oracles see exactly those faults. The evidence is that the arms’ ordering tracks fault character rather than generator sophistication: on the arithmetic-heavy rest-ncs both status-oracle arms score 0.0000 while EvoMaster kills 0.1714; on the validation-heavy rest-scs the LLM leads every arm. Two levels compose: oracle kind fixes the ceiling (status oracles cannot reach EvoMaster on value faults), and within a kind, volume decides how much of the reachable fault set is caught — which is why

the LLM’s 295 tests beat the leaner independent manual’s 147 where the faults are status-visible.

Three consequences follow. For practitioners, LLM generation is a breadth instrument that complements rather than replaces value-based oracles — it beats a human-style suite yet still missed more than half the faults EvoMaster’s regression oracles caught. For the field, the metric that ranks the LLM first over EvoMaster (volume) and the metric that ranks it below (ground-truth Recall) disagree on the same code, so the comparative claims currently in the literature are not measuring what their framing implies. For benchmark design, fault populations should be stratified by oracle-observability, because that single axis reorders the arms.

Our principal remaining limitation is that the independent manual baseline is an isolated agent session, not a human cohort; a human replication against the same 133 mutants is the highest-value next step, and our pipeline runs it without modification. Beyond it, the natural studies are an oracle-controlled comparison that fixes assertion kind and varies only the generator, a fault population stratified by oracle-observability, and the GAP-T ablation across model families. The pipeline, prompts, fault catalogs, and raw data are committed, and every number above regenerates from them by script.

References

- [1] M. Kim, T. Stennett, D. Shah, S. Sinha, and A. Orso, “Leveraging large language models to improve REST API testing,” in IEEE/ACM Int. Conf. on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2024.
- [2] T. Stennett, M. Kim, S. Sinha, and A. Orso, “AutoRestTest: A tool for automated REST API testing using LLMs and multi-agent reinforcement learning,” in IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2025.
- [3] M. Kim, S. Sinha, and A. Orso, “LlamaRestTest: Effective REST API testing with small language models,” in ACM Int. Conf. on the Foundations of Software Engineering (FSE), 2025.
- [4] M. Schäfer, T. Nguyen et al., “RESTifAI: An LLM workflow for reusable REST API testing,” in IEEE/ACM Int. Conf. on Software Engineering (ICSE), Demonstrations, 2026.
- [5] A. Arcuri, “RESTful API automated test case generation with EvoMaster,” ACM Transactions on Software Engineering and Methodology (TOSEM), vol. 28, no. 1, 2019.
- [6] K. Chen, Y. Zhang, J. Wang et al., “LogiAgent: Automated logical testing for REST systems with LLM-based multi-agents,” arXiv preprint, 2025.
- [7] M. Kim, Q. Xin, S. Sinha, and A. Orso, “Automated test generation for REST APIs: No time to rest yet,” in ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA), 2022.
- [8] Y. Liu, Y. Li, G. Deng et al., “Morest: Model-based RESTful API testing with execution feedback,” in IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
- [9] T. Tran, D. Le, H. Nguyen et al., “KAT: Dependency-aware automated API testing with large language models,” in IEEE Conf. on Software Testing, Verification and Validation (ICST), 2024.
- [10] A. Decrop, G. Perrouin, M. Papadakis, X. Devroey, and P.-Y. Schobbens, “You can REST now: Automated specification inference and black-box testing of RESTful APIs with large language models,” arXiv preprint, 2024.
- [11] A. Pereira, B. Lima, and J. P. Faria, “APITestGenie: Automated API test generation through generative AI,” in IEEE/ACM Int. Conf. on Automation of Software Test (AST), 2026.
- [12] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventrésque, “PIT: A practical mutation testing tool for Java (demo),” in ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA), 2016.
- [13] A. J. Offutt and R. H. Untch, “Mutation 2000: Uniting the orthogonal,” *Mutation Testing for the New Century*, pp. 34–44, 2001.